MA3K7 Assignment 3

Alexandros Sergiou (5505826)

1 Entry

I KNOW

From the assumption in the problem,

- Two players are given a pair of positive integers (n, s) corresponding to n heaps and s stones
- Players take turns placing one stone in one of n heaps (empty before the game starts)
- During the Nim phase, players take turns removing any positive number of stones from a single heap. The player who removes the final stone loses.

I assume that Player 1 makes the first placement and that turns continue alternating between the two phases. Therefore, if s is even, Player 1 begins the Nim phase, whereas if s is odd, Player 2 begins.

NOTATION

For brevity, I define:

- $H = (h_1, h_2, \ldots, h_n)$ corresponds to the resulting heaps, where h_i is the number of stones in heap i
- The Nim sum of the heaps, defined by $h_1 \oplus h_2 \oplus \cdots \oplus h_n$, where $\oplus$ denotes binary addition without carrying.

I WANT

I want to investigate this game and how it's played while exploring it with python, specifically I will:

- Simulate the game to understand how it works.
- Explore how different values of (n, s) affect the winning strategy.
- Investigate the usefulness of the Nim sum.
- Explore how changing the rules affects the outcome.

2 Attack

To begin, I used Python to simulate the game to help me visualise it. I followed a similar approach to the one seen in the lectures.

OBSERVE

The code helped me understand how the game works. However, since both players made random moves, the results did not tell me which player had a winning strategy.

IDEA

I then decided to improve my simulation by allowing both players to make optimal moves. To do this, I used a recursive function that considered every possible placement of a stone. The idea was that a position is winning if the current player can make a move that leaves the opponent in a losing position. If no such move exists, the position is losing. Once all the stones have been placed, the function uses the misère Nim winning criterion to determine the outcome of the second phase.

I then used this function to investigate how different values of (n, s) affected the winning player. In particular, I wanted to see whether the outcome depended on the parity of s or the relationship between n and s.

CHECK
STUCK

heaps (n)	stones (s)	Winner
2	3	Player 2
2	4	Player 2
5	4	Player 1
5	3	Player 2
6	5	Player 2
6	6	Player 2
7	6	Player 2
7	1	Player 1

Table 1: Winning players for different values of n and s, assuming optimal play.

Although I noticed some similarities between the results, I could not identify a general rule that explained all the cases I tested. The Python code identified which player could force a win, but it did not explain the strategies behind these outcomes.

IDEA

Rather than testing more values of (n, s), I decided to investigate whether the Nim sum could help explain the winning strategies.

ATTEMPT

I first considered the case $n = 2$ and $s = 4$. Since there are only two heaps, I wondered whether Player 2 could respond to each of Player 1's moves by placing a stone in the opposite heap. Suppose Player 1 places the first stone in heap 1. Player 2 can respond by placing a stone in heap 2, giving $(1, 1)$. Whatever heap Player 1 chooses next, Player 2 can again place a stone in the opposite heap. This guarantees the final configuration $(2, 2)$. Now since $2 \oplus 2 = 0$, and Player 1 begins the second phase, Player 2 can force a win under the last-stone-loses rule.

CHECK

I then tested whether this strategy would work for other values of s. For $n = 2$ and $s = 6$, Player 2 can again respond by placing a stone in the opposite heap, guaranteeing the final configuration $(3, 3)$. Since $3 \oplus 3 = 0$ and Player 1 begins the Nim phase, Player 2 can force a win. This suggested that the mirroring strategy might work for larger even values of s.

OBSERVE

When s is odd, Player 1 makes the final placement, so Player 2 cannot necessarily leave the heaps equal. However, Player 2 then begins the Nim phase, which suggests that a different strategy may be possible.

CHECK

To investigate this, I considered the case $n = 2$ and $s = 5$. Using the same strategy as before, Player 2 can ensure that after four placements the heaps are $(2, 2)$. Player 1 then places the final stone, giving either $(3, 2)$ or $(2, 3)$. In both cases, the Nim sum is

$$3 \oplus 2 = 1 \neq 0.$$

Since Player 2 begins the Nim phase, this is a winning position for Player 2.

NOTE

This suggests that the important factor is not simply whether the Nim sum is zero, but also which player begins the second phase. I then considered the case $n = 2$ and $s = 2$, to see whether the same strategy would work when fewer stones were available. Player 2 can place their stone in the opposite heap, producing $(1, 1)$, whose Nim sum is

$$1 \oplus 1 = 0.$$

However, unlike the cases $s = 4$ and $s = 6$, this is a winning position for Player 1 under the last-stone-loses rule. Player 1 can remove one stone, leaving Player 2 forced to take the final stone and lose. This made me realise that the usual Nim-sum criterion does not always apply under the last-stone-loses rule.

AHA

It appears we can use the nim sum to determine winning and losing positions under the last-stone-loses rule, provided at least one heap contains more than one stone. However, when every heap contains at most one stone and at least one stone remains, the winning criterion is

reversed. In this case, a Nim sum of zero indicates a winning position for the player whose turn it is. I shall check this observation using Python.

CHECK

To verify my observations, I wrote another Python function that examines all possible moves without using the Nim sum. I used this to determine winning and losing positions in the second phase, and compared the results with those obtained using my original Nim-sum-based algorithm.

Heaps	Nim sum	Recursive result	Outputs agree
$(2, 2)$	0	Losing	Yes
$(1, 1)$	0	Winning	Yes
$(3, 2)$	1	Winning	Yes
$(1, 1, 1)$	1	Losing	Yes
$(1, 1, 1, 1)$	0	Winning	Yes
$(3, 3)$	0	Losing	Yes

Table 2: Comparison of recursive results with the misère Nim winning criterion.

3 Solution

From my initial investigation, I found that Player 2 could force a win when $s > n$ and s was odd. However, after looking at the case $(n, s) = (5, 3)$, I realised that Player 2 could still force a win even when $s < n$. This made me consider whether Player 2 could guarantee a heap containing at least two stones through their placements, rather than relying on the values of n and s. This led me to the following result.

Claim 1. *If $s \geq 3$ is odd, then Player 2 has a winning strategy for any $n \geq 1$.*

Proof. Suppose Player 1 places the first stone in some heap. Player 2 can respond by placing a stone in the same heap. This guarantees that at least one heap contains more than one stone after the first phase, regardless of how the remaining stones are placed. Furthermore, since s is odd, there must be an odd number of odd-sized heaps. Therefore, the least significant binary digit of the Nim sum is 1, so the Nim sum cannot be zero. Under misère Nim, when at least one heap contains more than one stone, the player whose turn it is has a winning position if and only if the Nim sum is non-zero. Since Player 2 has ensured that at least one heap contains more than one stone, and we have shown that the Nim sum is non-zero, the final configuration is winning for the player about to move.

Finally, as s is odd, Player 2 begins the second phase from a winning position and can therefore force a win. □

CHECK

The condition $s \geq 3$ is important. When $s = 1$, Player 2 begins the second phase with only one stone remaining and is forced to remove it, losing under the last-stone-loses rule. Therefore, the result does not extend to $s = 1$.

Claim 2. *Suppose at least one stone remains and every heap contains at most one stone. Under the last-stone-loses rule, a position is winning for the player whose turn it is if and only if its Nim sum is zero.*

Proof. Since every non-empty heap contains exactly one stone, a player can only remove one stone per turn. Let k be the number of remaining stones. If k is even, the second player will take the final stone and lose, so the player moving first has a winning position. If k is odd, the first player will take the final stone and lose, so the position is losing for the player whose turn it is. Since each heap contains either 0 or 1 stone, the Nim sum is

$$h_1 \oplus \cdots \oplus h_n = \begin{cases} 0, & \text{if } k \text{ is even,} \\ 1, & \text{if } k \text{ is odd.} \end{cases}$$

Therefore, the position is winning for the player whose turn it is if and only if its Nim sum is zero. □

4 Review

CHECK

To check my findings, I used Python to test some additional values of (n, s) satisfying the conditions of my first claim. For each case, the program identified Player 2 as the winner, agreeing with my mathematical argument.

I also tested configurations where every heap contained at most one stone. The results

REFLECT

agreed with my second claim, including configurations containing empty heaps.

Initially, I relied on Python to compare different values of (n, s), but I soon realised that knowing which player won did not explain why they could force a win. Investigating the Nim sum helped me understand the strategies behind the results. Specifically, the case $(2, 2)$ showed me that I had to be careful when applying the usual Nim-sum criterion under the last-stone-loses rule. I also found that the mathematical arguments were more useful than checking individual games, since they allowed me to establish results for values of n and s that I had not tested.

One limitation of my Python approach was that the recursive functions became increasingly slow for larger values of n and s. This was because the program explored many possible placement sequences, sometimes calculating the same configurations more than once. Although I could have improved the efficiency by storing previously calculated results, I decided to focus on understanding the winning strategies mathematically. For example, my first claim establishes that Player 2 wins whenever $s \geq 3$ is odd, regardless of n, without having to

EXTENSION

compute every possible game.

I then decided to investigate how changing the last-stone-loses rule would affect the winning strategies. In particular, I considered the variation where the player who removes the final stone wins, essentially turning the second phase into standard Nim while keeping the first phase

NOTICE

unchanged.

I modified my Python code to compare the winning players under both variations, assuming optimal play. Some of the results are shown below.

n	s	Misère Nim	Standard Nim
2	1	Player 1	Player 2
2	2	Player 1	Player 2
2	3	Player 2	Player 2
2	4	Player 2	Player 2
3	1	Player 1	Player 2
3	2	Player 1	Player 2
3	3	Player 2	Player 2
3	4	Player 1	Player 1
4	2	Player 1	Player 2
4	3	Player 2	Player 2
4	4	Player 1	Player 2
4	5	Player 2	Player 2

Table 3: Comparison of the winning players under the two different rules.

OBSERVE

After comparing the results, I noticed that whenever $s > n$, the winning player remained the same under both variations. Since there are more stones than heaps, at least one heap must

contain more than one stone after the first phase, regardless of how the stones are placed. For these configurations, standard Nim and misère Nim have the same winning criterion. Therefore, every possible final configuration has the same winning status under both rules. Since the placement phase is unchanged, the players also have the same winning strategies during that phase, and so the winner of the complete game is unchanged.

EXTENSION

I then considered another variation, where players could remove at most two stones per turn during the second phase. I kept the first phase and the last-stone-loses rule unchanged.

OBSERVE

From the results, I noticed that restricting the number of stones that could be removed changed the winning player in several cases. For example, when $(n, s) = (3, 4)$, Player 1 wins under the original rules, whereas Player 2 wins when players can remove at most two stones per turn. I decided to investigate why this happens.

JUSTIFY

Suppose Player 1 places the first stone in heap 1. Player 2 can respond by placing a stone in the same heap, giving $(2, 0, 0)$. If Player 1 places the third stone in that heap, Player 2 can do the same, producing $(4, 0, 0)$. Alternatively, if Player 1 places the third stone in an empty heap, Player 2 can add another stone to the first heap, producing $(3, 1, 0)$.

I then considered whether these were losing positions for Player 1. From $(4, 0, 0)$, Player 1 can only leave $(3, 0, 0)$ or $(2, 0, 0)$. In either case, Player 2 can remove enough stones to leave exactly one stone remaining, forcing Player 1 to take the last stone. Similarly, from $(3, 1, 0)$, Player 1 can only leave $(2, 1, 0)$, $(1, 1, 0)$ or $(3, 0, 0)$, up to the order of the heaps. In each case, Player 2 can again leave exactly one stone remaining, forcing Player 1 to lose. Therefore, Player 2 can guarantee a losing position for Player 1 at the start of the second phase, and hence has a winning strategy when $(n, s) = (3, 4)$ under the restricted rules.

REFLECT

What I found particularly interesting was that restricting the possible moves changed the usefulness of the Nim sum. For example, $(4, 0, 0)$ has a non-zero Nim sum and is winning under the original rules. However, with a maximum removal of two stones, it becomes a losing position. This shows that the winning strategies depend not only on the final heap configuration, but also on which moves are permitted during the second phase.

Code

The code for this assignment, can be found on my GitHub page:
`https://github.com/Alex04cy/MA3K7-Assignment-3-`